

Convolutional Neural Network (CNN) Development for Image Classification

CIFAR-10 Dataset • TensorFlow/Keras • Data Augmentation • Feature Maps

Author: Tayyabah Rehman | June 2026

What This Project Does

This project designs, trains, and evaluates a Convolutional Neural Network (CNN) for multi-class image classification on the CIFAR-10 dataset. The implementation applies data augmentation to improve generalization, achieves 86.23% test accuracy with minimal overfitting, and includes feature map visualization to understand what the CNN learns at different depths.

Why It Matters

CNNs are the backbone of modern computer vision. Unlike dense networks, they exploit spatial structure in images — learning low-level edges in early layers and high-level object parts in deeper layers. This project demonstrates how architectural choices (convolution blocks, pooling, batch normalization, dropout) and data augmentation together yield a robust, well-generalizing image classifier, and how feature map visualization provides interpretability into the model's internal representations.

What You Will Learn

- How to load, normalize, and preprocess color image data for a CNN
- How to design a multi-block CNN architecture in TensorFlow/Keras
- How data augmentation reduces overfitting and improves generalization
- How to train and evaluate a CNN with training/validation monitoring
- How to extract and visualize intermediate feature maps for interpretability
- Why some classes (Cat, Dog) are harder to classify than others (Truck, Frog)

Dataset

CIFAR-10 is a benchmark dataset of 60,000 color images (32x32 pixels) across 10 classes. It is built into Keras and requires no manual download.

Property	Value
Source	tensorflow.keras.datasets.cifar10
Classes	10 (Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck)
Training Samples	50,000
Test Samples	10,000
Image Size	32 x 32 RGB (3 channels)

Model Architecture

Three convolutional blocks with increasing filter depth, followed by dense layers:

```
Input (32x32x3) → [Conv32 + BN + ReLU] x2 + MaxPool + Dropout(0.2)
                  → [Conv64 + BN + ReLU] x2 + MaxPool + Dropout(0.3)
                  → [Conv128 + BN + ReLU] x2 + MaxPool + Dropout(0.4)
                  → Flatten → Dense(256) + BN + Dropout(0.5) → Dense(10,
Softmax)
```

Parameter	Value
Total Parameters	816,938 (~3.12 MB)
Trainable Parameters	815,530 (~3.11 MB)
Non-trainable Parameters	1,408 (5.50 KB)

Data Augmentation

Augmentation is applied on-the-fly to training images using Keras ImageDataGenerator. This artificially expands the effective training set, reducing overfitting without collecting new data.

Technique	Configuration
Rotation Range	±15 degrees
Width / Height Shift	±10%
Horizontal Flip	Enabled

Zoom Range	±10%
Shear Range	±10%

Training Configuration

Hyperparameter	Value
Optimizer	Adam (learning_rate = 0.001)
Loss Function	Categorical Crossentropy
Batch Size	64
Epochs	50

Results

Metric	Value
Test Accuracy	86.23%
Test Loss	0.4277
Final Training Accuracy	86.08%
Final Validation Accuracy	86.23%
Overfitting Gap	0.0015 (Minimal)

Per-Class Performance Highlights:

- Best: Truck (97.0%), Frog (96.0%), Horse (95.9%)
- Worst: Cat (63.0%), Bird (76.8%), Dog (76.2%)
- Cat and Dog are frequently confused due to similar visual textures and features

Key Insights

- Data augmentation reduced the overfitting gap to just 0.15% between training and validation accuracy

- Spatial resolution decreases progressively: $32 \rightarrow 16 \rightarrow 8$ pixels across the three pooling layers
- Filter depth increases correspondingly: $32 \rightarrow 64 \rightarrow 128$, enabling hierarchical feature learning
- Early layers detect low-level features (edges, textures); deeper layers capture object-level parts
- Batch normalization and dropout work together to stabilize training and prevent overfitting

Project Structure

ML_Task_9_Tayyabah_Rehman.ipynb — Main notebook
cifar10_samples.png — Sample images from the dataset
augmentation_comparison.png — Original vs augmented image comparison
training_history.png — Training/validation accuracy and loss curves
confusion_matrix.png — Confusion matrix heatmap
per_class_accuracy.png — Bar chart of per-class accuracy
feature_maps.png — Feature maps from convolutional layers
deep_feature_maps.png — Average activation heatmaps per layer
sample_predictions.png — Test image predictions (correct/incorrect)
